

Classrooms

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“classrooms on campus”*
- ▶ *“as many activities as possible”*

Programming interpretation:

- ▶ This is interval scheduling with up to k simultaneous rooms.
- ▶ When more than k activities overlap, discard the one that ends latest.
- ▶ That keeps earlier finishing activities and leaves more room for the future.

Textbook connection: CP4 3.4 a. Classical Greedy

Core technique: Greedy interval scheduling with k rooms

- ▶ Activities have start and end times.
- ▶ At most k activities can be active at the same time.
- ▶ Activities ending exactly at another start still overlap in this problem.

- ▶ This is interval scheduling with multiple available rooms.
- ▶ When too many intervals overlap, keep the ones that end earliest.
- ▶ Priority queues let us remove finished intervals and reject the worst active interval.

- ▶ Sort activities by start time.
- ▶ Use a min-heap to expire active activities with $\text{end} < \text{current start}$.
- ▶ Tentatively accept the current activity.
- ▶ Use a max-heap of ending times to remove the active activity that ends latest if $\text{active} > k$.
- ▶ Count accepted activities.

- ▶ status marks whether an activity is active, removed, or finished.
- ▶ Lazy deletion handles heap entries whose status has changed.
- ▶ Removing the latest-ending active interval leaves the most room for future starts.

- ▶ At every start time, the accepted active set is feasible after any needed removal.
- ▶ If one active interval must be removed, dropping the one with latest end cannot reduce future availability compared with dropping an earlier-ending one.
- ▶ By exchange, the greedy active set can be transformed into an optimal active set after each step.

Complexity: $O(N \log N)$ time and $O(N)$ memory.

- ▶ Use $\text{end} < \text{start}$, not $\text{end} \leq \text{start}$, because equal endpoints overlap here.
- ▶ Python max-heap behavior is simulated with negative end times.

Source lines: classrooms.py:40-56

```
import sys
import heapq

def main():
    input = sys.stdin.buffer.readline

    n, k = map(int, input().split())

    activities = []

    for i in range(n):
        start, end = map(int, input().split())
        activities.append((start, end, i))

    activities.sort()
```

What this chunk does

- ▶ Each activity stores start, end, and original index.
- ▶ Sorting by start time lets the sweep process events chronologically.
- ▶ The original index is used to keep heap entries tied to status values.

Source lines: classrooms.py:58-72

```
min_heap = []  
max_heap = []  
status = [0] * n  
  
active = 0  
accepted = 0
```

What this chunk does

- ▶ The min-heap identifies accepted activities that have already ended.
- ▶ The max-heap identifies the currently active activity with latest ending time.
- ▶ status handles stale heap entries after an activity has been removed or finished.

Source lines: classrooms.py:74-82

```
for start, end, index in activities:
    while min_heap and min_heap[0][0] < start:
        old_end, old_index = heapq.heappop(min_heap)

        if status[old_index] == 1:
            status[old_index] = 2
            active -= 1
```

What this chunk does

- ▶ Before accepting a new activity, finished activities stop occupying rooms.
- ▶ The strict check `old_end < start` matches the statement that equal endpoints still overlap.
- ▶ Only status 1 activities affect the active-room count.

Source lines: classrooms.py:84-104

```
status[index] = 1

heapq.heappush(min_heap, (end, index))
heapq.heappush(max_heap, (-end, index))

active += 1
accepted += 1

if active > k:
    while max_heap:
        neg_end, remove_index = heapq.heappop(max_heap)

        if status[remove_index] == 1:
            status[remove_index] = 0
            active -= 1
            accepted -= 1
            break
```

What this chunk does

- ▶ The new activity is tentatively accepted first.
- ▶ If active exceeds k, the latest-ending active activity is removed.
- ▶ Removing the latest finisher is the greedy choice that preserves the most flexibility.

Source lines: `classrooms.py:106-109`

```
print(accepted)

main()
```

What this chunk does

- ▶ `accepted` has been updated throughout the sweep.
- ▶ Only the maximum count is required, not the actual classroom assignment.

Solution file: `practice_contests/day_3/classrooms.py`

Problem: <https://open.kattis.com/problems/classrooms>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.